



A Survey of Controller Placement Problem in SDN-IoT Network

Amel Abderrahmane¹ · Hamza Drid¹ · Amel Behaz¹

Received: 12 April 2024 / Accepted: 10 July 2024 / Published online: 25 July 2024
© The Author(s) 2024

Abstract

The Internet of Things (IoT) refers to the billions of intelligent physical devices connected to the Internet for collecting and sharing data. However, implementing IoT in large-scale industrial applications presents numerous challenges, including network management and scalability. These challenges encompass: complex network management tasks that are increasingly difficult to maintain, increased network resource usage, mobility, and high energy consumption. Software-defined networking (SDN) addresses these limitations by enforcing centralized control of all devices and leveraging a global network view. SDN is a networking paradigm that separates the control plane from the data plane, allowing managers to centralize the control of the network infrastructure. For large networks, such as IoT networks, multiple controllers are needed to manage the network efficiently. The Controller Placement Problem (CPP) involves the challenge of deploying the optimal number of controllers in a network while satisfying specific performance requirements such as latency, load balancing, and computation time. This paper provides an overview of recent research efforts addressing CPP issues in the SDN-IoT domain.

Keywords Internet of Things (IoT) · Software-Defined Networking (SDN) · Controller Placement Problem (CPP) · SDN-IoT

1 Introduction

According to Kevin Ashton [1], the Internet of Things is at the forefront of the second digital revolution. IoT refers to the interconnected network of physical devices embedded with sensors, software, and other technologies, enabling them to collect and exchange data. This facilitates higher levels of automation, efficiency, and real-time insights across various industries. However, one of the inherent complexities of IoT is its extensive scale. The large number of gateways and network devices required for high scalability also introduces significant management challenges.

Metcalf's Law [2] posits that as the number of interconnected devices increases, the value of communication networks grows exponentially. Consequently, IoT offers substantial benefits, as billions of interconnected items are expected to populate future networks. IoT is considered a

revolutionary technology due to the vast number of connected devices and the enormous volumes of data they generate. However, managing contemporary networks is challenging, and the existing centralized computing and storage models are increasingly insufficient. Future IoT solutions will necessitate novel technologies to analyze, filter, and aggregate data at the network level. The diverse nature of IoT data (in terms of type, semantics, frequency, location, and time) and the issues of integration, scalability, security, and privacy present significant obstacles to successful IoT deployments. Therefore, there is an urgent need for innovative technologies and approaches to enhance existing networks.

Software-defined networking (SDN) [3] has emerged as a promising solution to these challenges. SDN is a networking paradigm that separates the control plane from the data plane, allowing network administrators to program and manage network behavior dynamically and centrally via software applications. This decoupling facilitates direct network programming, offering numerous benefits [4, 5], such as simplified network management, improved network utilization efficiency, and enhanced support for network innovation. SDN supports a centralized control model wherein network intelligence is logically centralized in software-based

Authors equally contributed to this work.

✉ Amel Abderrahmane
a.abderrahmane@univ-batna2.dz

¹ LAMIE Laboratory, Department of Computer Science,
University of Batna 2, 05078 Batna, Algeria

controllers that maintain a global view of the network. The combination of IoT and SDN, referred to as SDN-IoT, leverages the strengths of both technologies to address network challenges effectively. IoT benefits from SDN's centralized control and programmability, which help manage the massive scale, diversity, and complexity of IoT networks. Conversely, SDN gains from IoT's extensive data generation capabilities, enhancing its ability to optimize network operations and innovate.

Initial SDN implementations, such as OpenFlow [4], assumed a single controller to manage the network. However, as SDN networks have evolved, it has become apparent that a single controller is insufficient to meet extensive management requirements [7]. To address this, logically centralized yet physically distributed multi-controller architectures, such as HyperFlow [8], Kandoo [9], and Onix [10], have been developed. These architectures aim to enhance network scalability and reliability while avoiding single points of failure [11].

One of the critical issues in multi-controller SDN networks is the Controller Placement Problem. The CPP involves determining the optimal number and placement of controllers in network why significantly impact network performances. This study provides an in-depth analysis of the challenges associated to the CPP in SDN-IoT networks. We present a comprehensive review of solutions proposed in the literature, categorizing them into mono-objective and multi-objective optimization solutions based on their performance improvements. Before discussing representative models and detailing the results of various solutions, we examine the fundamental factors of each optimization objective and their impact on network performance.

1.1 Summary of the Article's Contributions

- Present the first survey of the Controller Placement Problem in the SDN-IoT network, considering literature and industrial works.
- Provide an overview of the emergence of IoT from traditional networks to SDN-IoT environments.
- Present a classification of CPP approaches in IoT-SDN environments.
- Highlight the open research challenges and future research trends in the SDN-IoT environments.

This paper is structured as follows: Section 2 provides a comprehensive overview of the Internet of Things, Software-Defined Networking, SDN-IoT, and the Controller Placement Problem (CPP). Sections 3 to 4 discuss the current state of research, examining various methodologies, approaches, and solutions proposed in the literature to address the CPP within SDN-IoT environments. These sections also highlight practical implementations, case studies,

and potential advancements, offering a detailed analysis of their effectiveness and limitations. Section 5 concludes the paper by summarizing the key findings, emphasizing the critical importance of resolving the CPP for the efficient deployment of SDN-IoT networks, and proposing future research directions to guide ongoing and upcoming efforts in this evolving field.

2 Background

In this section, we introduce the basic architecture of IoT, the SDN, and the new term SDN-IoT. Following that, we examine the various difficulties encountered within the SDN-IoT network before presenting the comprehensive formulation and optimized objective of the controller placement problem.

2.1 Internet of Things

2.1.1 Introduction to IoT

Kevin Ashton [1] first proposed the idea of the Internet of Things in 1999 as a network of connected items that could each have a unique identification using RFID technology. Since then, the concept has evolved significantly, encompassing a vast array of interconnected devices that can transmit, receive, and link to each other using embedded sensors, software, and other technologies.

2.1.2 Definition of IoT

The IoT generally refers to a dynamic global network architecture characterized by self-configuring capabilities, built on widely acknowledged standards and communication protocols. It includes a wide range of networked devices connected to the internet, allowing them to transmit, receive, and interact seamlessly. These IoT devices, equipped with tiny computers, can communicate and operate within the global network, facilitating higher levels of automation, efficiency, and real-time insights across various industries. [12, 40, 41]

2.1.3 Architecture of IoT

The architecture of the IoT consists of the following layers Fig. 1.

- Perception layer: composed of physical objects that can detect physical quantities, [13] (such as heat, humidity, vibration, radiation, and others) and convert them into digital magnitudes. This information is processed, stored, and transmitted wirelessly to a sink or a network gate-

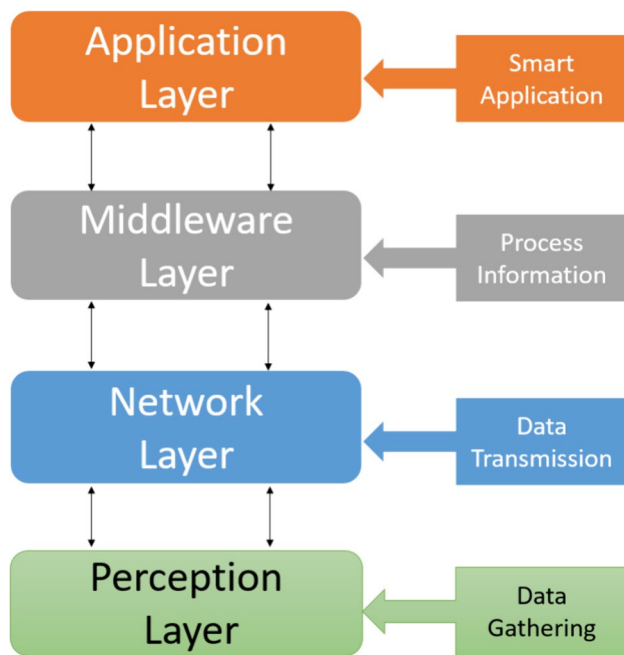


Fig. 1 IoT Architecture

way. This layer includes technologies such as: Wireless sensors [14], RFID [15], smartphones [16], wearables, intelligent homes [17], smart automobiles [18], and many other technologies.

- Network layer: it sends the digital information gathered in analog form from the physical world to a sink or a network gateway for further processing. [13]. Numerous technological advancements may be found in this context, including low-energy Bluetooth [14], Lorawan [16], wifi [19], Zigbee [20], and others.
- Middleware layer: Allows several Internet of Things devices inside a shared domain to communicate with one compatible device at a time. In order to do this, it transforms transmitted data into service information by pulling it out of different hardware components. The requested service's address and management services are included in this service information.
- Application layer: It acts as a user interface for information gathered from the perception layer, allowing users to modify the data to meet the needs of a particular domain and process it in a processing system. [42].

2.1.4 Challenges in IoT

Future IoT applications will face increasing demands and challenges as billions of connected devices become commonplace. These challenges encompass network management, network function virtualization, ubiquitous information access, resource utilization, energy management, security, and privacy. Addressing these challenges

necessitates the adoption of new technologies to enhance IoT network services, such as the innovative paradigm of SDN. SDN offers centralized control and dynamic network configuration, enabling the efficient management of extensive IoT deployments. It optimizes resource utilization by facilitating real-time allocation based on current demand, enhances security through uniform policy enforcement, and supports energy management by optimizing communication paths to minimize power consumption. Furthermore, SDN facilitates network function virtualization (NFV), allowing the deployment of virtualized network functions on standard hardware, thereby reducing costs and increasing flexibility. SDN's programmability enables adaptive routing and traffic management, ensuring reliable data delivery across geographically dispersed IoT devices. Consequently, SDN is an indispensable technology for addressing the scalability, efficiency, and security requirements of future IoT environment.[43].

2.2 Software-Defined Networking

2.2.1 Introduction to SDN

In the fast-changing field of network management, SDN has emerged as a groundbreaking approach, significantly simplifying and optimizing the control and operation of network infrastructure. Traditional networking paradigms, characterized by the tight coupling of the control plane and data plane, often lead to complex and inefficient network management. SDN addresses these challenges by introducing a centralized, programmable network architecture that enhances flexibility, scalability, and efficiency. [21].

2.2.2 Definition of SDN

SDN is a networking paradigm that decouples the control plane from the data plane in network devices, allowing for centralized management and dynamic configuration of the network. This separation is facilitated by an application programming interface (API) that enables communication between the SDN controller and the network devices. The controller, through a programmable interface on the switches, implements rules and manages traffic flows independently of the underlying infrastructure. SDN transforms traditional network management by providing a centralized control mechanism that oversees multiple network devices, ensuring efficient traffic flow and optimized network performance. This innovative approach not only simplifies network operations but also enhances the overall agility and responsiveness of the network.

2.2.3 Model of the SDN

The SDN architecture contains the following layers as depicted in Fig. 2:

- **Application layer:** Also known as the management plan [13], this layer enables applications to use northbound APIs to express network management policies that will be converted into commands by the controller and put into effect on network equipment directly through southbound APIs.
- **Control plan:** This segment has been separated from network devices and consolidated onto the SDN controller. This separation enables the translation of management policies from application-defined rules into directives for incoming flows. These rules are then injected into network routing devices using southbound APIs. The controller also facilitates the implementation of specific constraints on equipment, such as restricting the transmission of all ICMP packets except those with a destination IP address of X. Conversely, ICMP packets with a destination IP address of X are exclusively transmitted through port Y.
- **Data plan:** Depending on the connection module used in equipment for message routing from end to end, calls also have a transfer plan [13], which involves connecting equipment via wired cables or radio frequencies.
- **The APIs:** The SDN model uses four distinct APIs to communicate between its components: Northbound API, Southbound API, Eastbound API, and Outbound API.
- **The Northbound API** enables communication between the application layer and the control layer [22]. Through this interface, the controllers receive the application's quality of service requirements, which adds a layer of network abstraction and presents the network as a system [13].

- The Southbound API enables communication between the control plane and the data plane, allowing rules to be applied to routers, switches, wireless access points, and other devices directly [23].

2.2.4 Impact of SDN on IoT Network

The goal of SDN in an IoT is to provide a flexible and scalable infrastructure that can efficiently manage and control the large number of devices and data generated by IoT devices.

Some specific goals of SDN in an IoT network include:

- **Centralized Control:** SDN enables administrators to manage and control the network centrally, making it easy to configure, track, and manage IoT devices and their connectivity.
- **Dynamic Resource Allocation:** SDN enables the dynamic allocation of network resources according to the shifting needs of IoT devices. This guarantees optimal performance and efficient resource use.
- **Security:** SDN offers improved security functionalities through the centralized definition and enforcement of security policies by administrators. The technology facilitates the timely identification, prevention, and mitigation of potential risks throughout the entire IoT network.
- **Scalability:** SDN facilitates seamless scalability by accommodating the integration of additional IoT devices into the network. The system offers a flexible framework capable of handling the increasing number of devices while maintaining optimal performance and ease of management.
- **Traffic Optimization:** SDN enables intelligent traffic routing and load balancing in IoT network, ensuring efficient data transfer between devices, gateways, and cloud services.
- **Service Orchestration:** SDN facilitates service orchestration in an IoT network by providing a programmable infrastructure that dynamically adapts to changing service requirements.

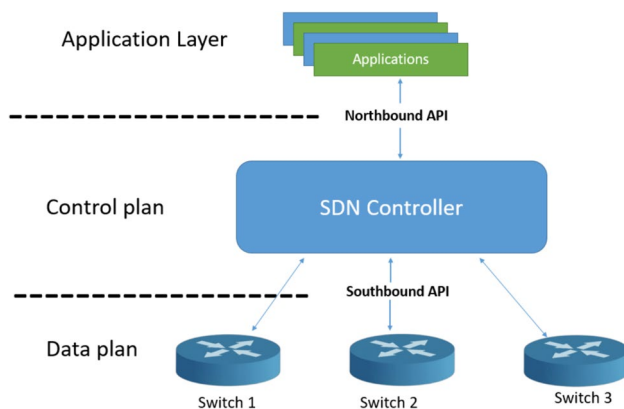


Fig. 2 SDN Architecture

2.3 SDN-IoT

2.3.1 Introduction to SDN-IoT

Qin et al. [24] developed an innovative framework to integrate SDN with IoT. This early work aimed to address the complexities of managing diverse IoT environments using the flexible and programmable features of SDN. By leveraging SDN's centralized control and dynamic resource allocation capabilities, the framework seeks to enhance the efficiency and scalability of IoT networks.

2.3.2 Definition of SDN-IoT

The proposed framework is designed to optimize heterogeneous architectures within SDN-IoT networks. It supports various devices, networks, and technologies within the IoT ecosystem, utilizing SDN for enhanced network management and optimization. Key features of the framework include the ability to dynamically allocate network resources, implement centralized security policies, and intelligently route traffic to ensure optimal performance. Additionally, the framework facilitates the seamless integration of new IoT devices and technologies, providing a versatile and scalable infrastructure for evolving IoT environments. This allows for improved interoperability and coordination among a wide range of IoT devices, ensuring that the network can adapt to changing conditions and demands while maintaining high levels of performance and security.

2.3.3 IoT Network Requirements for SDN

This section investigates several limitations in deploying IoT networks to enhance various urban activities. These limitations can be addressed by using SDN, effectively realizing the concept that “SDN-IoT.” We identify and differentiate these limitations as follows:

- **Efficient network management:** the proliferation of Internet of Things technologies will soon facilitate the connection of many objects to the internet, generating substantial volumes of data that necessitate effective and efficient processing. Managing these devices and their significant data volumes requires a robust approach. By adopting a global network perspective and implementing centralized data control, SDN technology facilitates the management and distribution of traffic flow to reduce latency and achieve load balancing within the network. Integrating SDN into IoT networks would enhance load balancing and improve the precision of message transportation, enabling optimal utilization of bandwidth resources.
- **Mobility:** The IoT will connect billions of devices over the next several years, and users must be able to use all of their devices' features and easily access the IoT network at any time and from any location. SDN technology enables efficient management of these devices, ensuring smooth handling of users' mobility [13].
- **Remedy the heavy use of network resources:** to enhance network efficiency, it is essential to develop a pre-established strategy for the desired quality of service and set up the network infrastructure to facilitate efficient traffic engineering. Excessive network utilization by users places a heavy burden on network equipment, diminishing its operational efficiency. Through the utilization of

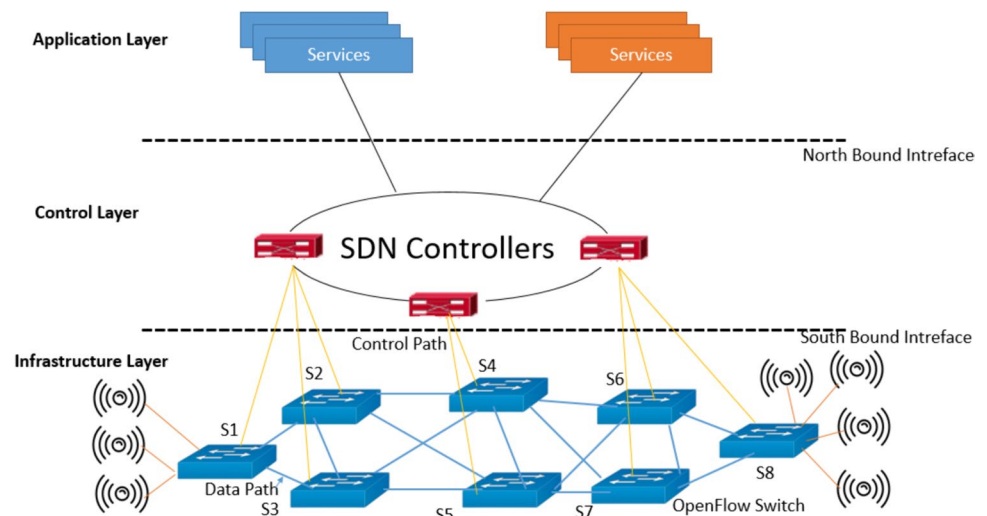
SDN, the SDN controller can determine the optimal path and establish the necessary rules for the ingress of data flow into the network equipment. This results in a reduction of network resource utilization and an enhancement of network efficiency.

- **Energy management optimization:** the connection of billions of devices to the IoT network would generate a substantial amount of data, necessitating the establishment of numerous data centers to handle this processing. Given the significant electrical energy requirements associated with operating data centers, it becomes imperative to construct data centers prioritizing energy efficiency. This can be achieved using intelligent energy management systems, effectively mitigating energy consumption. To establish energy-efficient IoT data centers based on SDN, SDN technology will facilitate the efficient routing of network traffic. This will be accomplished through the SDN controller, directing the network traffic to the appropriate server. Additionally, SDN technology will enable the activation or deactivation of data center equipment based on their energy consumption levels.

2.3.4 SDN-IoT Architecture

Three layers presented in (Fig. 3) make up the SDN based IoT architecture:

- **Infrastructure Layer:** it consists of various RFID [18] gadgets, wireless sensors [25], detection-capable smart phones [17], smart cars [14], and other gadgets. The connectivity of these devices to the access layer is established through wireless access points, base stations, and network gateways, facilitating their connection to the broadband backbone. Within the context of the SDN-IoT architecture, it is crucial to remember that the nodes involved primarily function as conduits for data transmission, thereby relinquishing the responsibility of higher-level decision-making and control to the SDN controller.
- **Control Layer:** the intermediate layer facilitates the connection between the application and infrastructure layers, allowing developers to construct IoT applications through accessible APIs. Additionally, it enables the direct implementation of rules onto network hardware to effectively manage the underlying infrastructure at a lower level. The SDN controller operates at this layer and allocates diverse hardware resources among the applications across different industries. Furthermore, it facilitates the routing of data with the requisite quality of service to end users situated in the network's core.
- **Application Layer:** to optimization of IoT applications, the application layer offers a complete separation from the underlying network transport and data connection layers. This is achieved by utilizing northbound APIs,

Fig. 3 SDN-IoT Architecture

allowing seamless interaction with the network infrastructure.

Overall, SDN-IoT networks provide a robust, secure, scalable, and efficient infrastructure that can support the diverse requirements of IoT applications and services. Since the first SDN-IoT framework proposal, numerous research papers have emerged. The increasing number of devices (nodes, controllers, and switches) in the network, underscores the critical importance of placing controllers in relation to routing nodes in an SDN-IoT-based design. This challenge is widely known in the literature as the Controller Placement Problem.

2.4 Controller Placement Problem (CPP)

It consists of finding the optimal controller placement to optimize one or more performance metrics, such as, latency, capacity of the controllers, load balancing and energy consumption. The controller placement problem seeks to answer two main questions:

- How many controllers should be used in the SDN-IoT network?
- What is the best location for each controller?

2.4.1 General formulation of CPP

The SDN-IoT network is modelled as a graph $G=(V, E, S)$ in which each node $v \in V$ represents an IoT device (router, switch, and any other component), and edge $e \in E$ is the physical connection that represents the communication link between these nodes. S represents the set of controllers in the network. In particular $k=|S|$ denotes the number of controllers.

According to Heller et al.'s research [26], the CPP is an NP-hard issue that was first examined to identify an optimal solution site. Since then, other research teams have worked to find a solution. The gap between controllers was considered a problem in research by Ishigaki et al. [27], who suggested that bringing the controllers closer together would resolve this issue.

St-Hilaire and Sallahi [28] determined the ideal site, amount, and type of controllers to use for a CPP. They also decided which connections should be made between each network component. Their model aims to reduce network costs while considering other constraints. Akbar Neghabi et al. [29] proposed a nature-inspired method for load balancing in SDN. [30] Introduced a heuristic technique for controller placement based on learning automata (LA). [31] Developed a novel routing protocol to reduce power consumption by heterogeneous devices.

2.4.2 Performance Impacted by CPP

Heller et al. [26] started by examining the effects of controller placement on network maximum and average delay. Other goals have been set, such as load balancing, reliability, and energy conservation. Multi-objective optimization has also proven to be a viable option when conflicts exist between multiple objectives.

- **Latency:** is the time between sending and receiving a data message. It measures how long a data packet takes to travel from one place to another. Several factors can affect latency, including network congestion, the distance between devices, and processing times at either end of the connection.
- **Load-Balancing:** distributes network traffic among several servers or other network devices. The goals of load-balancing are to optimize resource usage, increase

throughput, reduce response times, and prevent overload on any device.

- **Reliability:** is the extent to which a system can be relied upon to perform its intended function under given circumstances for a specific period. Reliability is crucial in various industries, including engineering, manufacturing, transportation, healthcare, and technology.
- **Energy-Saving:** involves consuming less energy to protect the environment and natural resources. Using energy-efficient equipment, reducing unnecessary energy consumption, and utilizing renewable energy sources are just a few ways to accomplish this. Energy conservation has a positive impact on the environment, reduces energy costs, and promotes sustainability.

3 Related Works

In this section, we comprehensively explore all existing solutions within this domain, systematically classifying them into two distinct subclasses based on the extent of performance enhancements: mono-objective and multi-objective optimization.

3.1 Mono-Objective

The solutions presented in this section are focused on enhancing a specific metric, namely latency (between switch-controller and controller-controller):

Keshari et al. [32] introduce an innovative hybrid Differential Evolution and Whale Optimization (DEWO) algorithm to solve the CPP in SDN for IoT-enabled smart cities. SDNs separate control logic from the hardware data plane, enabling centralized and programmable management of network devices, which is crucial for efficiently handling the vast number of IoT devices generating significant data in smart city environments. Traditional IoT networks face challenges such as access delays, security issues, and reliability concerns, which SDN-enabled networks can overcome. However, a single controller is insufficient for managing the extensive network of IoT devices, necessitating multiple controllers. This solution aims to optimize the placement of these controllers to enhance network performance.

The DEWO algorithm methodology begins with the initialization of population parameters for Differential Evolution (DE) and Whale Optimization (WO). The process involves extracting node locations based on longitude and latitude from the network topology and setting initial populations. Fitness functions are defined to minimize both switch-to-controller and controller-to-controller latencies. The DE component performs mutation and crossover operations to generate new candidate solutions, ensuring diversity and avoiding premature convergence. The

WO component refines these solutions by simulating the bubble-net hunting strategy of humpback whales, updating positions through spiral and shrinking mechanisms. These steps are iteratively repeated, with fitness evaluations, selection, crossover, mutation, and WO optimization, until the maximum number of iterations is reached or a satisfactory solution is found.

Results from MATLAB simulations on three different network topologies—TataNld, Deutsche, and Forthnet—demonstrate the DEWO algorithm's effectiveness. The algorithm consistently achieved lower average and maximum latencies compared to other metaheuristic algorithms like Genetic Algorithm (GA), Particle Swarm Optimization (PSO), and Firefly Algorithm (FFA). For instance, in the TataNld topology, DEWO reduced average latency by approximately 7.82% compared to PSO and 2.35% compared to FFA. Similar improvements were observed in the Deutsche and Forthnet topologies. The DEWO algorithm also maintained better scalability and performance stability as the number of controllers increased, demonstrating its robustness in handling large-scale networks. However, the DEWO algorithm has several limitations. The hybrid nature of the algorithm results in increased computational complexity and longer computation times, especially for very large networks. Parameter sensitivity is another issue, as the effectiveness of the DEWO algorithm depends on the appropriate selection of parameters such as crossover rate (Cr) and scaling factor (Fr). Additionally, the algorithm assumes static network topologies, which may not reflect the dynamic nature of real-world smart city environments where network topologies frequently change. The focus on latency minimization and single-objective optimization also limits the algorithm's applicability in scenarios requiring a comprehensive approach to network performance, considering factors such as energy efficiency, fault tolerance, and security. Addressing these limitations in future research will be essential to further enhance the DEWO algorithm's applicability and performance in real-world SDN-IoT networks for smart cities.

Ali et al. [33] present a comprehensive approach to address the CPP in SDN for IoT environments. The SDN paradigm decouples the control plane from the data plane, enabling centralized management of network devices. This separation is particularly advantageous for IoT networks, which involve numerous devices generating substantial data. In an SD-IoT setup, sensors frequently exchange data with controllers, and inappropriate placement of these controllers can significantly increase end-to-end (E2E) latency. The authors propose a novel clustering strategy leveraging the Analytical Network Process (ANP), a multi-criteria decision-making (MCDM) method, to optimize controller placements, improving network performance by reducing latency and balancing the load across controllers.

The methodology begins by modeling the SDN network as a graph and segmenting it into clusters. Each cluster's switches are identified, and criteria such as hop count, propagation latency, queuing latency, path computation latency, and link utilization are defined for evaluating the placement of controllers. The ANP model is used to rank switches within each cluster, determining the optimal locations for controllers. The process involves creating pairwise comparison matrices for each criterion, normalizing these matrices, and calculating eigenvectors to determine the priority of each switch. The final placements are refined by ensuring consistency through indices such as the Consistency Index (CI) and Consistency Ratio (CR), making sure the judgments are coherent.

Results from simulations conducted on four network topologies (OS3E, US_Net, Abilene, and Interoute) using Mininet and Matlab demonstrate that the proposed ANP-based clustering strategy significantly reduces E2E delay compared to the standard k-means algorithm. The authors found that the ANP method not only minimizes delays between switches and controllers but also reduces controller-to-controller (C2C) delays and communication overhead. For instance, in the Abilene topology, the total E2E delay was significantly reduced using the ANP-based approach compared to the k-means algorithm. The reduction in delay is attributed to the more effective placement of controllers as per the defined criteria, which ensures better allocation of switches to controllers. The proposed method also demonstrated a decrease in C2C delay. The correct positioning of controllers, considering multiple criteria, contributes to this reduction. The results indicated that the suggested approach has a shorter C2C delay in all topologies compared to k-means, especially as the number of clusters increases. The fairness index, representing the equitable distribution of switches among clusters, showed that the proposed method provided a fairer allocation of switches compared to k-means. A lower value of the fairness index indicates a smaller number of switches allocated to each cluster relative to the number of controllers, which is better achieved by the proposed approach. The proposed method exhibited lower communication overhead between switches and controllers (SW-CT) and among controllers (CT-CT) compared to the k-means algorithm. Lower communication overhead results in fewer packets exchanged, leading to less frequent interactions with controllers and reduced computational load on controllers.

Despite its advantages, the solution has limitations. The ANP model, with its detailed pairwise comparisons and consistency checks, increases computational complexity. This can result in longer computation times, especially for large networks with numerous nodes and controllers. The iterative nature of the process requires significant computational resources, which might not be practical for real-time

applications where quick decision-making is necessary. Although the proposed method shows good scalability, handling an increased number of controllers and nodes, there may still be challenges when applied to extremely large-scale networks. The performance of the method could degrade as network size grows beyond the tested topologies. The effectiveness of the ANP model depends on the appropriate selection of parameters for criteria such as hop count, propagation latency, and others. Incorrect parameter settings can lead to suboptimal solutions or slow convergence, requiring extensive experimentation and domain knowledge to fine-tune. The approach assumes static network topologies, which may not reflect the dynamic nature of real-world IoT networks where topologies frequently change due to mobility or varying load conditions. Adapting the model to handle dynamic and adaptive network topologies requires further research and development. The primary focus is on minimizing latency (both E2E and C2C) and balancing the load. While this is crucial, other important factors like energy efficiency, fault tolerance, and security are not explicitly addressed. A more comprehensive approach considering multiple performance metrics simultaneously would provide a more holistic solution to the CPP. The current implementation focuses on single-objective optimization, primarily minimizing latency. However, real-world network optimization often involves multiple conflicting objectives such as cost, reliability, and throughput. Extending the approach to support multi-objective optimization could enhance its applicability and effectiveness in diverse network scenarios.

In conclusion, the ANP-based clustering strategy proposed by Ali et al. offers a robust and efficient solution to the CPP in SDN-IoT networks, significantly reducing E2E and C2C delays and improving overall network performance. However, addressing the limitations mentioned above in future research will be essential to further enhance the solution's applicability and performance in real-world scenarios. Future work could explore dynamic network conditions, multi-objective optimization, and considerations for energy efficiency, fault tolerance, and security to provide a more comprehensive and adaptive solution for SDN-IoT networks.

3.2 Multi-Objective

The second part of the preposition encompasses all existing solutions that significantly enhance multiple aspects of performance:

Choumas et al. [35] delves into the integration of SDN with low-power IoT, highlighting the optimization of control traffic as crucial for minimizing energy consumption and maximizing bandwidth in IoT systems. The study addresses the challenge of selecting IoT sensors to host SDN controllers and mapping each sensor to a controller, both of which significantly affect the volume of control traffic.

The authors model this optimization problem using integer quadratic programming (IQP) and propose a set of heuristic algorithms to expedite the solution. They initially present a fast and simple heuristic algorithm, which is then extended to two iterative algorithms designed to offer improved performance at the cost of increased time complexity. The algorithms focus on minimizing the total control traffic, which includes both controller-to-switch (Ctr–Sw) and controller-to-controller (Ctr–Ctr) communications. Through simulations using network topologies from the Internet Topology Zoo collection and real-world experiments on the NITOS testbed, the results reveal that these heuristics achieve near-optimal performance with significantly less computation time than solving the IQP problem directly. The simple heuristic algorithm performs well, providing a good balance of performance and computational efficiency. The iterative algorithms, however, offer even better performance, particularly for larger networks, by refining the initial solution through a series of local optimizations. The study concludes that balancing the number and placement of controllers is critical for managing control traffic effectively. More distributed controllers reduce controller-to-switch traffic, whereas fewer, centralized controllers minimize controller-to-controller traffic. This balance is essential for optimizing energy consumption and bandwidth in low-power IoT networks. The authors note that as the number of controllers increases and the placement becomes more distributed, the controllers get closer to the switches, reducing the volume of Ctr–Sw traffic. Conversely, fewer and more concentrated controllers reduce the volume of Ctr–Ctr traffic. The proposed heuristic approaches, while practical, involve trade-offs between performance and computational complexity. They do not always achieve the absolute optimal solution but strike a balance that is practical for real-world applications. The study acknowledges that the dynamic nature of IoT environments requires adaptable solutions, and further enhancements are suggested to improve these heuristics, particularly focusing on minimizing control delay. The authors provide insights and directions for future research, including exploring other centrality metrics, improving the balance between distributed and centralized controller placement, and enhancing the adaptability of the solutions to varying network conditions. These improvements aim to further reduce control traffic and enhance the overall performance of SDN-based IoT networks.

Anh Khoa Tran et al. [36] explores optimizing controller placement in SDN for IoT networks using a submodularity-based approach. The authors address the challenges of efficient controller placement in distributed and dynamic IoT networks. The methodology involves developing an optimization framework using submodularity theory to handle various aspects of controller placement in distributed networks. The submodularity theory helps in formulating and

addressing different controller placement problems, ranging from basic to complex use cases. The framework includes heuristic algorithms based on the submodularity concept to find near-optimal solutions for different scenarios.

The submodularity framework simplifies the controller placement problem into five main use cases:

- **Maximum Covered Submodular Set Problem (P1):** This involves placing a set of controllers that can control the entire network while maximizing a submodular function. A greedy incremental algorithm with an approximation ratio is used for this problem. The objective is to find a subset of nodes that maximizes the network utility while adhering to the constraint of having a limited number of controllers.
- **Maximum Covered Submodular Set Problem with Given Requirement Set of Controllers (P2):** This problem extends P1 by including a given set of existing controllers in the network. The solution involves transforming the original function to account for the pre-existing controllers. The goal is to expand the current set of controllers by adding new ones to maximize the network utility while maintaining the existing controllers.
- **Maximization Control Submodular Problem (P3):** This problem considers a connected graph of all controllers and aims to place controllers such that the sub-graph is connected and maximizes a submodular function. The algorithm ensures that the placement results in a connected network of controllers, which is essential for maintaining network integrity and performance.
- **Maximization Network Quality Factor Submodular Problem (P4):** This involves selecting a subset of controller locations to maximize the network quality factor, which includes constraints like hop count or latency measurements. The objective is to ensure that the chosen controllers provide the best possible network performance metrics, such as minimizing latency and maximizing throughput.
- **Maximization Submodular Problem with Budget Constraints (P5):** This problem introduces an economic aspect by considering deployment costs and budget constraints for placing controllers. The goal is to optimize the placement of controllers within a given budget, ensuring cost-effective network management.

The proposed methodology involves detailed steps for each use case, utilizing submodular optimization principles. The Nemhauser's algorithm is used extensively to approximate solutions to these NP-hard problems, providing a balance between computational efficiency and solution quality. For example, in P1, the greedy algorithm incrementally selects nodes that maximize the utility function until the desired number of controllers is reached.

In P2, the algorithm is extended to include pre-existing controllers, starting with the given set and then adding new controllers that maximize the network utility. P3 and P4 further refine the placement strategy by ensuring connectivity and optimizing network quality factors, respectively. P5 incorporates budget constraints, transforming the problem to handle economic considerations effectively.

The results from extensive simulations show that the proposed submodularity-based algorithms outperform baseline methods regarding execution time, number of controllers, and network latency. The simulations involved creating a physical network using Zipf's law [37] to generate random nodes and links, evaluating the execution time, and measuring performance aspects like the number of controlled nodes and average latency.

For instance, in the simulation environment, a physical network with 200 nodes was created, and the algorithms were evaluated based on their ability to control the network efficiently. The results demonstrated that the submodularity-based approach achieved near-optimal solutions in significantly less time compared to the optimal solutions obtained using the Gurobi solver [34]. The average latency and number of controllers were also evaluated, showing that the proposed method effectively reduced network latency and required fewer controllers compared to baseline methods.

The paper discusses the limitations of the current approach, noting that while the submodular approach effectively matches the controller placement problem, it does not fully consider the relationship between controllers in the control channel. Future work aims to combine submodularity theory with other methods to account for control plane costs and enhance the framework's applicability to real-world scenarios.

In conclusion, the authors present a promising framework for solving the SDN controller placement problem in IoT networks, emphasizing the effectiveness of submodularity theory in providing flexible and efficient solutions for various network scenarios. The methodology, supported by extensive simulations, demonstrates significant improvements in execution time, network latency, and the number of controllers required, making it a viable approach for practical implementation in distributed IoT networks.

Sikander Hans et al. [40] focuses on optimizing the placement of controllers in SDN-IoT networks to reduce latency and improve load balancing. The authors propose using an Enhanced Sunflower Optimization (ESFO) algorithm combined with a Pareto Optimal Controller (POCO) placement tool. This method is compared with existing methods such as PASIN, hybrid SD, and PSO.

The methodology includes the implementation of an adaptive fuzzy controller placement using the ESFO algorithm and POCO tool. The ESFO algorithm mimics the natural movement of sunflowers towards the sun to find optimal

solutions, while the POCO tool helps design the controllers and nodes to solve the controller placement problem effectively. The fuzzy logic system used includes four stages: fuzzifier, knowledge base, defuzzifier, and fuzzy inference engine. The fuzzifier converts input data into fuzzy sets, the knowledge base stores the fuzzy rules, the defuzzifier converts fuzzy sets into a crisp output, and the fuzzy inference engine processes the fuzzy rules to derive an output. The adaptive fuzzy controller is designed to reduce the communication burden among controllers by placing multiple controllers at different locations. This reduces the network delay caused by the limited control capability of a single controller. The proposed method optimizes controller placement by considering load balancing, delay constraints, and failure tolerance. The objective is to find an optimal solution for delay constraints, controller selection, and adaptive fuzzy controller placement by balancing the load and reducing propagation latency. The population-based, nature-inspired ESFO algorithm imitates the path taken by sunflowers as they approach the sun. The algorithm includes the following steps: initializing the population of flowers, calculating the objective function for all sunflowers, estimating the orientation vector for each plant, removing plants farthest from the sun, calculating steps for each plant, pollinating the best plants, and evaluating new individuals. The objective function maximizes the density and minimizes the distance to the controllers. The algorithm ensures that the optimized number of controllers is identified based on high density and minimum distance.

The POCO placement tool is used to design controllers and nodes to solve the controller placement problem. It calculates optimal results and resilience against controller failures by considering latency and load constraints. The tool helps in finding the best controller placement in the network by minimizing the maximum latency between nodes and controllers, even in failure scenarios.

The results indicate that the proposed method achieves significant improvements in latency and load balancing compared to existing methods. With two controllers, the proposed method obtains 400 miles as average latency, which is 22.2% smaller than PSO, 76.9% lesser than hybrid SD, and 91.89% lesser than PASIN. The study includes detailed comparison tables and graphical representations of controller placement, node failure scenarios, and controller imbalance under different conditions.

The paper discusses the main limitations related to the complexity of controller placement in large-scale networks and the challenges in achieving minimal latency under varying network conditions. The proposed method addresses these challenges but requires further testing in real-world scenarios to validate its effectiveness. In conclusion, the proposed ESFO algorithm combined with the POCO tool offers an efficient solution for controller placement in SDN-IoT

networks. The method provides improved load balancing and reduced latency compared to existing approaches. Future work includes testing the proposed strategy in realistic scenarios and considering additional performance metrics like energy consumption and response time to enhance the quality of service in IoT networks.

The research by Khera et al. [44] investigates enhancing the performance of wide-area Cellular IoT (CIoT) networks by integrating SDN with IoT networks. The focus is on finding the optimal placement of SDN controllers and evaluating SDN clustering to boost efficiency. The study proposes an Un-Supervised Machine-Learning (US-ML) approach based on silhouette distance and gap statistics to determine the optimal number of controllers. The Partition Around Medoids (PAM) method is used to allocate these controller locations effectively.

The study begins by identifying the critical need for improving the performance of CIoT-based wide area networks (WANs). It suggests combining Low-Power Wide Area Networks (LPWAN) and CIoT networks to enhance performance. The research introduces a hybrid combination of SDN and IoT networks, which has gained popularity in industrial applications such as smart cities, production control, smart homes, and inventory management. The proposed method involves using US-ML techniques to determine the optimal number of SDN controllers. This is achieved by analyzing silhouette distance and gap statistics, followed by the PAM approach to allocate the controller locations.

In detail, the study generates topological data from a gml file using MATLAB, deriving the adjacency matrix, node coordinates, and level matrix. The PAM method is applied to find the best controller placements by evaluating cluster quality and network latencies. This method minimizes the sum of dissimilarities between points and their nearest medoid, which helps in reducing communication distances between sensor nodes and controllers. The effectiveness of the proposed method is validated through simulations, where average and worst-case latencies are calculated to ensure robust performance under different network conditions. The study introduces a modified worst-case latency calculation that better reflects real-world scenarios, enhancing the robustness of the results.

The study presents various results based on simulations using the proposed US-ML and PAM methods. It evaluates the performance of different LPWAN technologies, including NB-IoT, LoRa, and SigFox, in terms of coverage and energy efficiency. NB-IoT is found to have the longest battery life and most effective energy management, making it highly suitable for long-term IoT applications. Effective placement of SDN controllers significantly reduces communication distances between sensor nodes, thereby improving network performance. The performance of the proposed method is validated by evaluating both average

and worst-case latencies. The modified worst-case latency calculation performs better under different network conditions compared to traditional methods. The quality of clustering is assessed using the silhouette method. The results indicate that the majority of clusters have positive silhouette values, suggesting a better distribution of clusters within the network and improved network performance. The silhouette values range from -0.5 to 1 , with fewer than 10% of clusters showing negative values, indicating a high-quality clustering outcome.

Despite the promising results, the study acknowledges several limitations. The use of random distance measures in the methodology leads to probabilistic outcomes, meaning the results may vary. To address this, future simulations should use fixed distance measures for network formation to achieve more consistent results. Another limitation is the challenge of implementing LoRa in WAN architecture, despite its optimal maximum range. This complexity makes it difficult to deploy effectively in real-world scenarios. Additionally, the study recognizes that the method uses random distance measures, leading to probabilistic outcomes that may vary. To address this, future research should focus on using fixed distance measures for network formation to achieve more consistent results.

In conclusion, the study highlights the potential of combining SDN with IoT networks to enhance the effectiveness of WAN networks by improving data control. The proposed US-ML based silhouette PAM method for optimal SDN controller placement demonstrates significant improvements in network performance. Future research should focus on testing the performance of LoRaWAN and NB-IoT networks under reduced power consumption scenarios to further validate the findings. Overall, the integration of SDN with CIoT networks using advanced machine learning techniques for controller placement holds considerable promise for improving various industrial applications.

Table 1 below summarizes all the solutions (methods used, goals, performances improved, etc.)

3.2.1 Performance Metrics

The following table (Table 2) summarizes the metrics used by the different proposed solutions to evaluate their CPP algorithms.

4 Future Research Works

This section highlights important research issues that should be considered when examining the CPP in the SDN-IoT. We reviewed the CPP-related solutions put forth by various researchers in light of latency, switch assignment, Delay E2E, communication cost inflation, energy

Table 1 All the existing solutions to the problem placement controllers in SDN-IoT network

Title	Methods	Performance improved	Simulation	Goal	Critique
SDN Controller Placement in IoT Networks: An Optimized Submodularity-Based Approach [36]	Optimization with two algorithms (N'enhancer's Algorithm and the search tree algorithm)	Latency Execution time control channel	Physical network using Zipf's law 200 nodes and 10 controllers compared with optimal and closest- distance	Controller placement and number of controllers	-Fixed the maximum number of controllers (if the number needed is greater than the fixed)Neglecting some of the mentioned parameters like the control channel and other quality of services
An Intelligent way for optimal controller placements in software-defined-IoT networks for smart cities. [32]	Hybrid Differential Evolution and Whale Optimization (DEWO) Algorithm	Latency Balance the switch loads	Matlab2019b TataNld, Forthnet, Deutsche topologies switches between 50 to 150 controllers between 1 to 40	Controller placement	-Neglecting the number of controllers (when the number increases, the latency other metrics mentioned
SDN Controller Placement and Switch Assignment for Low Power IoT [35]	Integer quadratic programming (IQP). And Heuristic algorithm	Switch assignment Minimizes the required bandwidth Control traffic	Internet Topology Zoo collection. Switches less or equal to 30	Controller placement and a number of controllers	heuristic solutions are inefficient for the placement solution. Neglecting latency as an important metric
An Effective Approach for Controller Placement in Software-Defined Internet-of-Things (SD-IoT) [33]	Analytical Network Process (ANP) and multi-criteria decision-making (MCDM) scheme	Deley (end-to-end and controller-to-controller) fair allocation of switches communication overhead	Real Internet topologies. Abilene, US_Net, OS3E, Interoute topology. Compared with K-means	Controller placement	Neglecting the number of controllers and the parameters mentioned in the first
Controller Placement in Software Defined Internet of Things Using Optimization Algorithm. [40]	Adaptive fuzzy controller. And enhanced sunflower optimization (ESFO) algorithm. And Pareto Optimal Controller Placement Tool (POCO)	Load balance reduced number of controllers average latency and delay	The MATLAB/Simulink implements the system with two controllers	Controller placement and number of controllers	Proposal of a number of controllers is not efficient
Enhancing Performance of Wide Area CIoT SDN by US-ML Based Optimum Controller Placement [44]	Un Supervised Machine Learning (US-ML) Approach Silhouette Distance Analysis Gap Statistics Partition Around Medoids (PAM) Clustering Method Modified Worst-Case Latency Calculation	Latency Energy Clustering Quality	MATLAB Data Source: gml file, 14 nodes representing different cities	Controller placement and number of controllers	The use of random distance measures leads to probabilistic outcomes, requiring fixed distance measures for consistent results Implementing LoRa in WAN architecture is challenging despite its optimal maximum range, complicating real-world deployment

Table 2 number of the metrics improved with each solution

	Latency	Load-Balancing	Control Traffic	Others
Keshari et al. [32]	X			
Ali et al. [33]	X			
Choumas et al. [35]			X	switch assignment- increase bandwidth
Tran et al. [36]	X			-Number of controllers -time execution -Control channel
Hans et al. [40]	X	X		Reduced number of controllers and average latency and delay
Khera et al. [44]	X			Energy Clustering Quality

costs, and different goal-related functions. There are still several obstacles to be explored. By addressing various issues and proposing potential research directions, we aim to stimulate researchers delving into this topic.

- **Network performance:** performance is an essential parameter for evaluating an SDN-IoT network. It can be measured in terms of throughput, latency, and response time. A high-performance network must efficiently manage the data generated by connected objects.
- **Security:** is another critical parameter to evaluate in an SDN-IoT network. The data generated by connected objects can be sensitive and must be protected against malicious attacks. The network must have security mechanisms such as authentication, confidentiality, and data integrity.
- **Scalability:** is another important parameter for evaluating an SDN-IoT network. The number of connected objects can increase rapidly, which requires the ability to scale the network rapidly.
- **Traffic Management:** is essential for evaluating an SDN-IoT network. Connected objects can generate large amounts of data, leading to network congestion if traffic is improperly handled.
- **Flexibility:** is also necessary when evaluating an SDN-IoT network. The network must adapt to changes in the IoT environment and user needs.
- **Interoperability:** is also essential for evaluating an SDN-IoT network. Connected objects can come from different suppliers and use other communication protocols. The network must be able to support these different protocols to ensure efficient communication between related entities.
- **Reliability:** is a key consideration for evaluating an SDN-IoT network. The network must ensure consistent and dependable performance, even in the face of hardware failures, network congestion, or other issues. High reliability is essential to maintain trust and functionality in critical applications.

5 Conclusions

As SDN-IoT networks rapidly evolve, using a single controller to manage multiple IoT-enabled switches can be inefficient. Therefore, multiple controllers are needed to manage all switches in the SDN IoT network. We are therefore faced with the problem of determining their location and quantity. Use different methods to solve controller placement issues. However, as the number of controllers increases, the effectiveness of these methods decreases due to previous changes. CPP is a current research hotspot on SDN architecture behavior. Various types of research have been conducted to address CPP issues and try to improve performance metrics (latency, reliability, cost, etc.). Due to the inherent complexity of the CPP and its NP-Hard nature, existing solutions cannot fully satisfy all fundamental requirements for various network types with absolute precision.

This article mainly studies the concepts and architecture of IoT and SDN as well as the concept of SDN-IoT. We then explore the idea of the CPP and highlight the importance of considering it in the context of SDN for the IoT. The CPP can be categorized into two main groups: mono-objective solutions and multi-objective solutions. Moreover, we have discussed each module and its mathematical formulation and provided a thorough analysis of CPP solutions, noting any drawbacks. Table 1 lists the disadvantages of each CPP solution and can be used to determine the most effective approach to solving CPP problems. Finally, we list some research topics that researchers can pursue further. The main purpose of this survey is to provide a comprehensive overview of the CPP, identify gaps in the existing literature, and stimulate discussion on possible future research directions. The survey highlights the challenges posed by SDN-IoT architecture and positions CPP as one of the most essential topics in this network. It is observed that multiple controllers are required to achieve the scalability and reliability of

SDN-IoT. Determining the optimal number and location of controllers is critical to improving network performance. A review of all CPP solutions reveals the existence of multiple approaches based on specific objective functions and constraints.

Future research directions should focus on developing more sophisticated algorithms for controller placement that leverage artificial intelligence and machine learning to predict network conditions and optimize controller locations dynamically. Additionally, exploring the integration of edge computing with SDN-IoT frameworks can significantly enhance data processing efficiency and reduce latency, making the network more responsive and resilient. Practical applications of these advancements include smart cities, where optimized controller placement can ensure seamless connectivity and efficient management of IoT devices in urban environments. Moreover, the advent of 5G technology and beyond will provide new opportunities to enhance SDN-IoT infrastructures by offering higher bandwidth and lower latency, further supporting real-time applications such as autonomous vehicles and industrial automation. By addressing the CPP with emerging technologies, we can create more robust, scalable, and efficient networks capable of meeting the demands of future IoT applications. This ongoing research will pave the way for innovative solutions that can transform various industries and improve overall network performance and reliability.

Funding Not applicable.

Data availability Not applicable.

Declarations

Conflict of Interest The authors declare that they have no competing interests.

Ethical Approval Not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Ashton K (2009) That 'internet of things' thing. *RFID J* 22(7):97–114
- Metcalfe's Law. <https://en.wikipedia.org/wiki/Metcalfe>.
- Kreutz D, Ramos FM, Verissimo PE, Rothenberg CE, Azodolmolkly S, Uhlig S (2014) Software-defined networking: a comprehensive survey. *Proc IEEE* 103(1):14–76
- McKeown N, Anderson T, Balakrishnan H, Parulkar G, Peterson L, Rexford J, Turner J (2008) OpenFlow: enabling innovation in campus networks. *ACM SIGCOMM Comput Commun Rev* 38(2):69–74
- Lin YD, Lin PC, Yeh CH, Wang YC, Lai YC (2015) An extended SDN architecture for network function virtualization with a case study on intrusion prevention. *IEEE Network* 29(3):48–53
- Liu J, Jiang Z, Kato N, Akashi O, Takahara A (2016) Reliability evaluation for NFV deployment of future mobile broadband networks. *IEEE Wirel Commun* 23(3):90–96
- Guo Z, Su M, Xu Y, Duan Z, Wang L, Hui S, Chao HJ (2014) Improving the performance of load balancing in software-defined networks through load variance-based synchronization. *Comput Netw* 68:95–109
- Tootoonchian A, Ganjali Y (2010) Hyperflow: a distributed control plane for openflow. In *Proceedings of the 2010 internet network management conference on research on enterprise networking* (Vol. 3, pp. 10–5555)
- Hassas Yeganeh S, Ganjali Y (2012) Kandoo: a framework for efficient and scalable offloading of control applications. In *Proceedings of the first workshop on hot topics in software defined networks* (pp. 19–24).
- Koponen T, Casado M, Gude N, Stribling J, Poutievski L, Zhu M, Shenker S (2010) Onix: a distributed control platform for large-scale production networks. In *9th USENIX symposium on operating systems design and implementation (OSDI 10)*
- Guo Z, Liu R, Xu Y, Gushchin A, Walid A, Chao HJ (2017) STAR: Preventing flow-table overflow in software-defined networks. *Comput Netw* 125:15–25
- Chen J, Liu Y, Zhang Y (2024) Advances in IoT-enabled smart environments. *Sensors* 24(5):1234–1250
- Zemrane H, Baddi Y, Hasbi A (2018) SDN-based solutions to improve IoT: survey. In *2018 IEEE 5th international congress on information science and technology (CiSt)* (pp. 588–593). IEEE.
- Satam P, Pacheco J, Hariri S, Horani M (2017) Autoinfofotainment security development framework (ASDF) for smart cars. In *2017 International conference on cloud and autonomic computing (ICCAC)* (pp 153–159). IEEE.
- Pal D, Funilkul S, Charoenkitkarn N, Kanthamanon P (2018) Internet-of-things and smart homes for elderly healthcare: an end user perspective. *IEEE Access* 6:10483–10496
- Rolik J, Lens JE, Dewoolkar MM, Weller TM (2018) Effects of soil characteristics on passive wireless sensor interrogation. *IEEE Sensors J* 18(8):3454–3460
- Jadhav KB, Chaskar UM (2017) Design and development of smart phone based ECG monitoring system. In *2017 2nd IEEE international conference on recent trends in electronics, Information & Communication Technology (RTEICT)* (pp. 1568–1572). IEEE.
- Zhang JF, Wen CJ (2017) The university library management system based on radio frequency identification. In *2017 10th international congress on image and signal processing, BioMedical Engineering and Informatics (CISP-BMEI)* (pp. 1–6). IEEE.
- Kim J, Song J (2018) A secure device-to-device link establishment scheme for LoRaWAN. *IEEE Sens J* 18(5):2153–2160
- Nishikori S, Kinoshita K, Tanigawa Y, Tode H, Watanabe T (2017) A cooperative channel control method of ZigBee and WiFi for IoT services. In *2017 14th IEEE annual consumer communications & networking conference (CCNC)* (pp. 1–6). IEEE.
- Priscilla O (2010). Top-down network design.
- Bonfim MS, Dias KL, Fernandes SF (2019) Integrated NFV/SDN architectures: a systematic literature review. *ACM Comput Surv (CSUR)* 51(6):1–39

23. Kunz T, Muthukumar K (2017) Comparing openflow and NETCONF when interconnecting data centers. In 2017 IEEE 25th international conference on network protocols (ICNP) (pp. 1–6). IEEE.
24. Qin Z, Denker G, Giannelli C, Bellavista P, Venkatasubramanian N (2014) A software defined networking architecture for the internet-of-things. In 2014 IEEE network operations and management symposium (NOMS) (pp. 1–9). IEEE.
25. Frolik J, Lens JE, Dewoolkar MM, Weller TM (2018) Effects of soil characteristics on passive wireless sensor interrogation. *IEEE Sens J* 18(8):3454–3460
26. Heller B, Sherwood R, McKeown N (2012) The controller placement problem. *ACM SIGCOMM Comput Commun Rev* 42(4):473–478
27. Ishigaki G, Gour R, Yousefpour A et al (2017) Cluster leader election problem for distributed controller placement in sdn. In: *GLOBECOM 2017–2017 IEEE Global Communications Conference*. IEEE, p. 1–6
28. Sallahi A, St-Hilaire M (2014) Optimal model for the controller placement problem in software defined networks. *IEEE Commun Lett* 19(1):30
29. Milan ST, Rajabion L, Ranjbar H, Navimipour NJ (2019) Nature inspired meta-heuristic algorithms for solving the load-balancing problem in cloud environments. *Comput Oper Res* 110:159–187
30. Polat H, Polat O, Cetin A (2020) Detecting DDoS attacks in software-defined networks through feature selection methods and machine learning models. *Sustainability* 12(3):1035
31. Kharkongor C, Chithralekha T, Varghese R (2016) A SDN controller with energy efficient routing in the internet of things (IoT). *Procedia Comput Sci* 89:218–227
32. Keshari SK, Kansal V, Kumar S (2021) An intelligent way for optimal controller placements in software-defined-IoT networks for smart cities. *Comput Ind Eng* 162:107667
33. Ali J, Roh BH (2022) An effective approach for controller placement in software-defined internet-of-things (SD-IoT). *Sensors* 22(8):2992
34. <https://www.gurobi.com/>
35. Choumas K, Giatsios D, Flegkas P, Korakis T (2020) SDN controller placement and switch assignment for low power IoT. *Electronics* 9(2):325
36. Tran AK, Piran MJ, Pham C (2019) SDN controller placement in IoT networks: an optimized submodularity-based approach. *Sensors* 19(24):5474
37. Newman ME (2005) Power laws, Pareto distributions and Zipf's law. *Contemp Phys* 46(5):323–351
38. Wang A, Wu J (2018). Policy and resource orchestration in software-defined networks. In 2018 IEEE 4th international conference on collaboration and internet computing (CIC) (pp. 203–206). IEEE.
39. Hans S, Ghosh S, Kataria A, Karar V, Sharma S (2022) Controller placement in software defined internet of things using optimization algorithm. *CMC-Comput Mater Continua* 70(3):5073–5089
40. Gupta P, Kumar R, Singh S (2024) Integration of AI and IoT: a comprehensive review. *IEEE Internet Things J* 11(1):56–72
41. Khan M, Ali Z, Rahman A (2024) Blockchain-enhanced IoT architecture for secure data management. *Internet Things J* 25(3):345–360
42. Herencsar N, Soltan A et al (2023) Internet of things: a comprehensive overview on protocols, architectures, technologies, simulation tools, and future directions. *Energies* 16(8):3465
43. Zhu Q, Wang L, Li Y (2023) Software-defined networking for internet of things: a comprehensive review. *J Netw Comput Appl* 123:45–67
44. Khera A, Kurmi US (2023) Enhancing performance of wide area CIIoT SDN by US-ML based optimum controller placement. *Res Rep Comput Sci*

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.